

64-123 SOFTWAREENGINEERING I

an der *uni-hamburg*

bei Prof. Dr. Manfred Exemplé im WiSe 2020/21

Übung 1

Ausarbeitung von

<i>Max Mustermann</i>	<i>Maxine Musterfrau</i>
<i>1234567</i>	<i>2345678</i>

FAKULTÄT
FÜR MATHEMATIK, INFORMATIK
UND NATURWISSENSCHAFTEN

Zusammenfassung

In diesem Aufsatz betrachten wir den Satz von Erdős-Kac, ein fundamentales Resultat der analytischen Zahlentheorie, das die Verteilung der Anzahl der Primfaktoren einer natürlichen Zahl beschreibt. Wir werden den Satz präzise formulieren, seine Bedeutung erläutern und einen Beweisansatz diskutieren. Schließlich betrachten wir einige Anwendungen und Verallgemeinerungen dieses Satzes.

1 Einführung

Die Zahlentheorie untersucht die Eigenschaften der natürlichen Zahlen $\mathbb{N} = \{1, 2, 3, \dots\}$, insbesondere die Primzahlen. Eine interessante Fragestellung ist, wie sich die Anzahl der Primfaktoren einer zufällig ausgewählten natürlichen Zahl verhält. Der Satz von Erdős-Kac liefert hierzu eine erstaunliche Antwort und stellt eine Verbindung zwischen Zahlentheorie und Wahrscheinlichkeitstheorie her.

2 Formulierung des Satzes von Erdős-Kac

Sei $n \in \mathbb{N}$ eine natürliche Zahl und $P(n)$ die Anzahl der Primfaktoren von n , wobei jeder Primfaktor mehrfach gezählt wird. Für eine natürliche Zahl n bezeichnen wir mit $\omega(n)$ die Anzahl der verschiedenen Primfaktoren von n und mit $\Omega(n)$ die Anzahl der Primfaktoren von n unter Berücksichtigung ihrer Vielfachheit.

Der Satz von Erdős-Kac besagt, dass die zufällige Variable $\frac{\omega(n) - \log \log n}{\sqrt{\log \log n}}$ für große n asymptotisch standardnormalverteilt ist. Genauer:

Theorem 2.1 (Erdős-Kac). Für jede reelle Zahl z gilt:

$$\lim_{N \rightarrow \infty} \frac{1}{N} \left| \left\{ n \leq N : \frac{\omega(n) - \log \log n}{\sqrt{\log \log n}} \leq z \right\} \right| = \Phi(z),$$

wobei $\Phi(z)$ die Verteilungsfunktion der Standardnormalverteilung ist.

3 Beweisidee

Der Beweis des Satzes von Erdős-Kac basiert auf der Methode der Momente und dem zentralen Grenzwertsatz. Wir skizzieren hier die Hauptschritte des Beweises.

Aufgabe 1.1: Beweisidee des Satzes von Erdős-Kac. Zeigen Sie, dass die zufällige Variable $\frac{\omega(n) - \log \log n}{\sqrt{\log \log n}}$ asymptotisch standardnormalverteilt ist.

1. Zunächst zeigen wir, dass $\mathbb{E}[\omega(n)] = \log \log n + \mathcal{O}(1)$. Dies folgt aus der Tatsache, dass die Wahrscheinlichkeit, dass eine Zahl n von einem Primfaktor p geteilt wird, ungefähr $1/p$ ist, und die Summe der Kehrwerte der Primzahlen divergiert logarithmisch.
2. Weiterhin zeigen wir, dass die Varianz von $\omega(n)$ asymptotisch $\log \log n$ ist. Dies kann durch ähnliche Überlegungen wie bei der Erwartung gezeigt werden.
3. Schließlich wenden wir den zentralen Grenzwertsatz an. Da $\omega(n)$ als Summe unabhängiger Indikatorvariablen geschrieben werden kann, konvergiert die standardisierte Variable $\frac{\omega(n) - \mathbb{E}[\omega(n)]}{\sqrt{\text{Var}[\omega(n)]}}$ gegen die Standardnormalverteilung.

4 Anwendungen und Verallgemeinerungen

Der Satz von Erdős-Kac hat zahlreiche Anwendungen in der Zahlentheorie. Eine wichtige Anwendung ist die Abschätzung der Anzahl der Zahlen mit einer bestimmten Anzahl von Primfaktoren in einem gegebenen Intervall. Zudem lässt sich der Satz verallgemeinern auf andere arithmetische Funktionen.

Aufgabe 1.2: Anwendung des Satzes von Erdős-Kac. Bestimmen Sie die Anzahl der natürlichen Zahlen $n \leq N$, die genau k verschiedene Primfaktoren haben.

Wir verwenden den Satz von Erdős-Kac, um zu zeigen, dass die Anzahl der natürlichen Zahlen $n \leq N$ mit genau k verschiedenen Primfaktoren asymptotisch gegeben ist durch:

$$\frac{N}{\sqrt{2\pi \log \log N}} \exp\left(-\frac{(k - \log \log N)^2}{2 \log \log N}\right).$$

Dies folgt aus der Tatsache, dass die Verteilung der Anzahl der Primfaktoren einer zufällig gewählten Zahl asymptotisch normal ist.

5 Fazit

Der Satz von Erdős-Kac ist ein bemerkenswertes Resultat der analytischen Zahlentheorie, das eine tiefe Verbindung zwischen der Verteilung der Primfaktoren von natürlichen Zahlen und der Normalverteilung aufzeigt. Durch die Methode der Momente und den zentralen Grenzwertsatz lässt sich zeigen, dass die Anzahl der Primfaktoren asymptotisch normalverteilt ist, was zahlreiche Anwendungen und Verallgemeinerungen ermöglicht.

Zusammenfassung

In diesem Aufsatz untersuchen wir die Verteilung der Primzahlen und stellen den Satz von Dirichlet über Primzahlen in arithmetischen Progressionen vor. Wir werden den Satz präzise formulieren, einige zentrale Ideen des Beweises skizzieren und Anwendungen sowie Verallgemeinerungen diskutieren.

6 Einführung

Die Verteilung der Primzahlen ist ein zentrales Thema der Zahlentheorie. Ein bedeutendes Resultat in diesem Bereich ist der Satz von Dirichlet, der die Existenz unendlich vieler Primzahlen in bestimmten arithmetischen Progressionen garantiert. Wir beginnen mit der Formulierung dieses Satzes.

7 Der Satz von Dirichlet

Aufgabe 1.3: Der Satz von Dirichlet. Zeigen Sie, dass es für zwei teilerfremde Zahlen a und d unendlich viele Primzahlen der Form $a + nd$ gibt, wobei $n \in \mathbb{N}$.

Theorem 7.1 (Dirichlet). Seien a und d zwei teilerfremde natürliche Zahlen. Dann gibt es unendlich viele Primzahlen der Form $a + nd$, wobei n eine nichtnegative ganze Zahl ist.

Beweis. Die Beweisidee basiert auf analytischen Methoden, insbesondere auf der Anwendung von L -Funktionen und Charakteren modulo d .

1. Zunächst definieren wir den Dirichletschen L -Funktion $L(s, \chi)$ für einen Charakter χ modulo d :

$$L(s, \chi) = \sum_{n=1}^{\infty} \frac{\chi(n)}{n^s}.$$

2. Wir zeigen, dass $L(1, \chi) \neq 0$ für den nicht-trivialen Charakter χ modulo d .
3. Schließlich verwenden wir die Nichtverschwindung von $L(1, \chi)$, um zu zeigen, dass die arithmetische Progression $a + nd$ unendlich viele Primzahlen enthält.



8 Anwendungen und Verallgemeinerungen

Der Satz von Dirichlet hat wichtige Anwendungen in der Zahlentheorie. Eine direkte Anwendung ist die Bestimmung der Verteilung der Primzahlen in verschiedenen arithmetischen Progressionen. Zudem lassen sich ähnliche Resultate für andere algebraische Strukturen herleiten.

Aufgabe 1.4: Anwendung des Satzes von Dirichlet. Bestimmen Sie die Anzahl der Primzahlen in der arithmetischen Progression $a + nd$ für große n .

Wir verwenden den Satz von Dirichlet, um zu zeigen, dass die Anzahl der Primzahlen in der arithmetischen Progression $a + nd$ für $n \leq x$ asymptotisch gegeben ist durch:

$$\frac{x}{\varphi(d) \log x},$$

wobei φ die Eulersche Phi-Funktion ist.

Aufgabe 1.5: Verallgemeinerung. Betrachten Sie eine Verallgemeinerung des Satzes von Dirichlet auf quadratische Formen. Zeigen Sie, dass es unendlich viele Primzahlen der Form $n^2 + 1$ gibt.

Diese Verallgemeinerung ist ein offenes Problem in der Zahlentheorie. Obwohl wir wissen, dass es unendlich viele Primzahlen der Form $4n + 1$ und $4n + 3$ gibt, bleibt die Frage nach Primzahlen der Form $n^2 + 1$ ungelöst. Es wird vermutet, dass es unendlich viele solche Primzahlen gibt, aber ein Beweis steht noch aus.

9 Fazit

Der Satz von Dirichlet ist ein fundamentales Resultat in der Theorie der Primzahlen und hat weitreichende Konsequenzen für die Verteilung der Primzahlen in arithmetischen Progressionen. Die Anwendung analytischer Methoden, insbesondere der L -Funktionen, ermöglicht tiefere Einblicke in die Struktur der Primzahlen und bietet Ansatzpunkte für weitere Forschungen.

Zusammenfassung

In diesem Aufsatz untersuchen wir den Algorithmus von Dijkstra zur Bestimmung der kürzesten Wege in einem Graphen. Wir werden die Funktionsweise des Algorithmus erläutern, seine Komplexität analysieren und den Beweis der Korrektheit führen.

10 Einführung

Der Algorithmus von Dijkstra ist ein fundamentaler Algorithmus in der Graphentheorie und der Informatik, der zur Berechnung der kürzesten Wege von einem Startknoten zu allen anderen Knoten in einem gewichteten Graphen verwendet wird. Wir beginnen mit einer formalen Beschreibung des Problems und einer Übersicht über den Algorithmus.

11 Formale Beschreibung

Gegeben sei ein Graph $G = (V, E)$ mit einer Gewichtsfunktion $w : E \rightarrow \mathbb{R}^+$. Wir wollen für einen Startknoten $s \in V$ die kürzesten Wege zu allen anderen Knoten in V finden.

12 Der Algorithmus von Dijkstra

Aufgabe 1.6: Algorithmus von Dijkstra. Implementieren Sie den Algorithmus von Dijkstra zur Berechnung der kürzesten Wege in einem Graphen.

Der Algorithmus von Dijkstra funktioniert wie folgt:

1. Initialisiere die Distanzen: $d(s) = 0$ und $d(v) = \infty$ für alle $v \in V \setminus \{s\}$.
2. Setze alle Knoten als unbesucht. Erstelle eine Menge Q , die alle Knoten enthält.
3. Solange Q nicht leer ist:
 - (a) Wähle einen Knoten u aus Q mit der kleinsten Distanz $d(u)$.
 - (b) Entferne u aus Q .
 - (c) Für jeden Nachbarn v von u :
 - i. Berechne die alternative Distanz $alt = d(u) + w(u, v)$.
 - ii. Wenn $alt < d(v)$, setze $d(v) = alt$ und aktualisiere den Vorgänger von v .

Aufgabe 1.7: Beweis der Korrektheit. Zeigen Sie, dass der Algorithmus von Dijkstra korrekte kürzeste Wege berechnet.

1.7.1 Zeigen Sie, dass der Algorithmus von Dijkstra die Knoten in aufsteigender Reihenfolge ihrer kürzesten Entfernung vom Startknoten verarbeitet.

Wir zeigen dies durch Induktion. Für den Startknoten s ist die Behauptung offensichtlich, da $d(s) = 0$ die kleinste Entfernung ist. Angenommen, die Behauptung gilt für alle Knoten mit $d(v) \leq d(u)$. Der nächste Knoten, der ausgewählt wird, hat die kleinste verbleibende Distanz, somit $d(u)$.

1.7.2 Zeigen Sie, dass die kürzeste Distanz von s zu jedem Knoten v korrekt berechnet wird.

Angenommen, es gibt einen kürzeren Pfad zu einem Knoten v als den durch den Algorithmus gefundenen. Dann müsste es einen Knoten u auf diesem Pfad geben, der nicht korrekt verarbeitet wurde, was einen Widerspruch zu unserer vorherigen Annahme liefert. Daher ist der Algorithmus korrekt.

13 Komplexitätsanalyse

Aufgabe 1.8: Komplexität. Bestimmen Sie die Zeitkomplexität des Algorithmus von Dijkstra.

Die Zeitkomplexität des Algorithmus von Dijkstra hängt von der Implementierung der Prioritätswarteschlange ab:

- Mit einer einfachen Warteschlange beträgt die Komplexität $\mathcal{O}(|V|^2)$.
- Mit einem binären Heaps beträgt die Komplexität $\mathcal{O}((|V| + |E|) \log |V|)$.
- Mit einem Fibonacci-Heaps beträgt die Komplexität $\mathcal{O}(|E| + |V| \log |V|)$.

14 Fazit

Der Algorithmus von Dijkstra ist ein effizienter Algorithmus zur Berechnung der kürzesten Wege in einem Graphen. Durch die Analyse und den Beweis der Korrektheit haben wir seine Bedeutung und Anwendungsmöglichkeiten unterstrichen. Die Wahl der richtigen Datenstruktur kann die Effizienz des Algorithmus erheblich beeinflussen.